

DBMS Unit 1 - Introduction to Database System

UNIT - I: INTRODUCTION TO DATABASE SYSTEM

MSBTE K-Scheme | Diploma CO | SY - Database Management System

1.1 DATABASE CONCEPTS

1. Data

Data is a collection of raw, unprocessed facts and figures that have no meaning by themselves until they are processed. Data can be in the form of numbers, text, images, sounds, or symbols.

Example: 21, "Rahul", "Pune" — these are raw facts.

When data is processed, organized, and given context, it becomes **information**. Example: "Rahul, age 21, lives in Pune" is information.

2. Database

A **database** is an organized collection of interrelated data stored together in a systematic way, so that it can be easily accessed, managed, retrieved, and updated.

Characteristics of a database: - Data is stored in a structured format (tables, rows, columns). - Data is related/interlinked logically. - Redundancy (duplication) of data is minimized. - Multiple users can access the data simultaneously. - Data is shared across various applications.

Example: A college database storing student records, faculty records, course details, and results.

3. Database Management System (DBMS)

A **DBMS** is software that enables users to define, create, maintain, and control access to the database. It acts as an interface between the user/application programs and the database.

Functions of DBMS: - Defining structure (schema) of the database - Storing and retrieving data efficiently - Providing security and authorization - Maintaining data integrity and consistency - Supporting concurrent access (multiple users at once) - Providing backup and recovery mechanisms

Examples of DBMS software: MySQL, Oracle, SQL Server, PostgreSQL, MS Access.

4. File System Vs DBMS

Basis	File System	DBMS
Data Redundancy	High - same data may be duplicated in multiple files	Minimized through normalization
Data Consistency	Difficult to maintain due to redundancy	Easy to maintain
Data Sharing	Difficult; data is scattered in different files	Easy; centralized data can be shared
Data Security	Very limited security features	Strong security, authorization, and access control
Data Integrity	No built-in constraints	Enforced through integrity constraints
Backup & Recovery	Not efficiently supported	Efficient backup and recovery mechanisms
Concurrent Access	Not supported properly, may cause data inconsistency	Supported through concurrency control techniques
Query Processing	No query language, needs custom programs	Supports query languages like SQL
Cost	Low cost	Higher cost (software, hardware, trained staff)
Examples	COBOL/C file handling programs	MySQL, Oracle, SQL Server

5. Applications of DBMS

1. **Banking** - customer information, accounts, transactions, loans
2. **Railway/Airline Reservation** - ticket booking, schedules, seat availability
3. **Education** - student records, results, library management
4. **Hospital Management** - patient records, doctor details, billing
5. **E-commerce** - product catalogs, orders, customer data, payments
6. **Telecommunication** - call records, billing, customer data
7. **Manufacturing/Inventory** - stock, supply chain, sales
8. **Social Media** - user profiles, posts, connections
9. **Government** - census data, tax records, election data

6. Data Abstraction

Since a database system's users need efficient access without concerning themselves with complex internal storage details, DBMS provides **data abstraction** by hiding certain details of how data is stored and maintained. It is achieved through **three levels**:

a) Physical Level (Internal Level) - The lowest level of abstraction.
- Describes **how** data is actually stored on the storage device (files, indexes, blocks). - Concerned with data structures and access methods.

b) Logical Level (Conceptual Level) - Describes **what** data is stored and **what relationships** exist among the data. - Database administrators use this level to decide the overall logical structure. - Users at this level do not need to know physical storage details.

c) View Level (External Level) - The **highest** level of abstraction. - Describes only a **part** of the entire database relevant to a particular user. - Provides security by hiding data that is not needed by a specific user. - Multiple views can exist for the same database.

View Level	(End Users)
Logical Level	(DBA)
Physical Level	(Storage)

7. Data Independence

Data independence is the capacity to change/modify the schema at one level without affecting the schema at the next higher level. There are two types:

a) Physical Data Independence - The ability to modify the **physical schema** (storage structure, file organization) without altering the logical schema. - Example: Changing storage device or file organization method does not affect application programs.

b) Logical Data Independence - The ability to modify the **logical (conceptual) schema** without affecting the external schema (views) or application programs. - Example: Adding a new field/table to the database should not affect existing application programs. - Logical data independence is **harder to achieve** than physical data independence, since application programs are heavily dependent on the logical structure.

8. Database Schema

A **schema** is the overall design/structure of the database, described in terms of a data model. It defines the variables (tables, fields, relationships) but **not** the actual data values.

Types of Schema (corresponding to 3-level abstraction):

1. **Physical Schema** - describes database design at physical level (storage & access methods)
2. **Logical Schema** - describes database design at logical level (tables, attributes, relationships, constraints)
3. **View Schema/Sub-schema** - describes different views for different users

Schema vs Instance: - **Schema** = the design/structure (does not change frequently) — like the blueprint of a building. - **Instance (Database State)** = the actual data stored at a particular moment (changes every time data is inserted/updated/deleted).

9. Codd's Rules

Dr. E.F. Codd proposed **12 rules** (numbered 0 to 12) that a database must satisfy to be considered a **truly relational database (RDBMS)**.

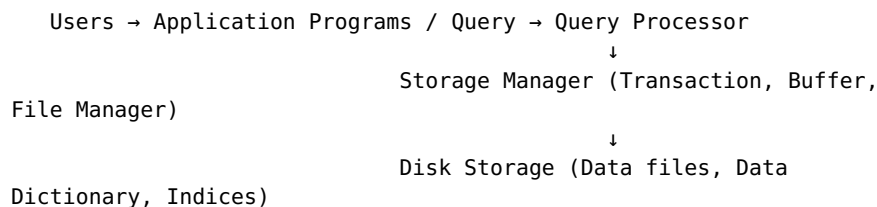
Rule No.	Rule Name	Description
Rule 0	Foundation Rule	The system must qualify as relational, as a database, and as a management system

Rule 1	Information Rule	All information must be represented as data values in tables (rows & columns)
Rule 2	Guaranteed Access Rule	Every data value must be accessible using table name + primary key + column name
Rule 3	Systematic Treatment of NULL values	NULL values must be handled consistently, representing missing/inapplicable data
Rule 4	Active/Dynamic Online Catalog	Database structure (metadata) must be stored in tables and be queryable using the same query language
Rule 5	Comprehensive Data Sub-language Rule	The system must support at least one language (e.g., SQL) for data definition, manipulation, and control
Rule 6	View Updating Rule	All views that are theoretically updatable must be updatable by the system
Rule 7	High-level Insert, Update, Delete	Data must be manipulable (insert/update/delete) as a set, not just one row at a time
Rule 8	Physical Data Independence	Application programs must remain unaffected by changes in physical storage
Rule 9	Logical Data Independence	Application programs must remain unaffected by changes in logical structure
Rule 10	Integrity Independence	Integrity constraints must be stored in the catalog, not in application programs
Rule 11	Distribution Independence	The system's operation should not be affected whether the database is distributed or centralized
Rule 12	Non-subversion Rule	If a low-level language allows row-at-a-time access, it must not bypass integrity rules

10. Overall Structure of DBMS

The DBMS structure can be divided into several components:

1. **Query Processor** - interprets user queries and converts them into instructions
 - DML Compiler, DDL Interpreter, Query Optimizer
2. **Storage Manager** - interacts with the file system to store and retrieve data
 - Authorization & Integrity Manager
 - Transaction Manager
 - File Manager
 - Buffer Manager
3. **Disk Storage** - actual physical storage
 - Data files
 - Data dictionary (metadata)
 - Indices
4. **Users of DBMS:**
 - **Database Administrator (DBA)** - manages the entire database
 - **Application Programmers** - write programs using DML
 - **Sophisticated Users** - interact using query language directly
 - **Naive/End Users** - use pre-written application interfaces (forms)



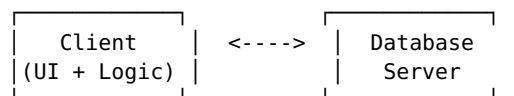
1.2 ARCHITECTURE OF DATABASE

Two-Tier Architecture

In two-tier architecture, the application is divided into **two layers**:

1. **Client Tier** - contains the user interface and application logic
2. **Server Tier (Database Tier)** - contains the DBMS

The client communicates **directly** with the database server using query language statements (e.g., via ODBC/JDBC).



Advantages: - Simple and fast for small applications - Direct communication reduces complexity

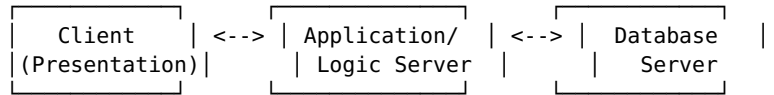
Disadvantages: - Not scalable for large number of users - Business logic mixed with UI on the client side (harder to maintain) - Security is lower since the client interacts directly with the database

Three-Tier Architecture

Three-tier architecture divides the system into **three layers**, adding a middle layer between client and database:

1. **Presentation Tier (Client/UI Layer)** - handles user interface, display of data
2. **Application/Logic Tier (Middle Layer/Server)** - contains business logic, processes requests, and communicates with the database. This is where the actual application program resides.

3. **Database Tier (Data Layer)** - stores and manages the actual data



Advantages: - Better scalability - can support many clients - Improved security - client never directly accesses the database - Business logic centralized, easier to maintain/update - Better performance through load balancing at the middle tier

Disadvantages: - More complex to design and implement - Increased network communication (extra layer) may add slight delay

Comparison - Two-tier vs Three-tier:

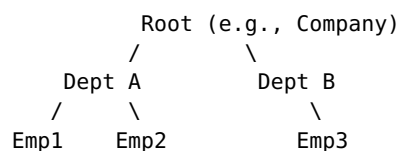
Basis	Two-Tier	Three-Tier
Layers	Client, Server	Client, Application Server, Database Server
Communication	Direct client-to-database	Via middle application layer
Security	Lower	Higher
Scalability	Limited	High
Maintenance	Harder (logic in client)	Easier (logic centralized)
Example	MS Access-based apps	Web applications (browser-server-DB)

1.3 DATA MODELS

A **data model** is a collection of conceptual tools/concepts used to describe data, data relationships, data semantics, and constraints.

1. Hierarchical Data Model

- Organizes data in a **tree-like structure** with parent-child relationships.
- Each parent (higher level) can have multiple children, but each child has **only one parent** (one-to-many relationship).
- The topmost node is called the **root**.



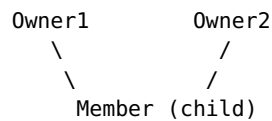
Advantages: - Simple and easy to understand for naturally hierarchical data - Fast data access since relationships are predefined - Data integrity maintained through parent-child links

Disadvantages: - Complex to implement many-to-many relationships - Insertion/deletion anomalies - difficult to insert a child without a parent - Not flexible - rigid structure, difficult to reorganize - Data redundancy may occur

Example System: IMS (Information Management System by IBM)

2. Network Data Model

- An extension of the hierarchical model.
- Data is represented as a **graph**, where a child (called “member”) can have **multiple parents (owners)**.
- Supports **many-to-many** relationships using the concept of “sets.”



Advantages: - Supports many-to-many relationships, more flexible than hierarchical - Faster data access via multiple paths - Better data representation of complex relationships

Disadvantages: - Complex structure - difficult to design and maintain - Requires knowledge of the physical structure to navigate (navigational model) - Any structural change requires changes in application programs

Example System: IDMS (Integrated Database Management System)

3. Relational Data Model

- Proposed by **Dr. E. F. Codd** in 1970.
- Data is organized in the form of **tables (relations)** consisting of **rows (tuples)** and **columns (attributes)**.
- Most widely used data model today.

Key Terms: | Term | Meaning | |—| | **Relation** | A table | | **Tuple** | A row/record in the table | | **Attribute** | A column/field in the table | | **Domain** | The set of allowable values for an attribute | | **Degree** | The number of attributes (columns) in a relation | | **Cardinality** | The number of tuples (rows) in a relation | | **Primary Key** | A unique attribute (or set) that identifies each tuple uniquely | | **Foreign Key** | An attribute referencing the primary key of another table, used to establish relationships |

Example Table - STUDENT

Roll_No (PK)	Name	Branch	Marks
1	Aditi	CO	85
2	Rahul	CO	78
3	Sneha	IF	90

Advantages: - Simple, easy to understand (tabular format) - Data independence (both physical and logical) - Supports powerful query languages like SQL - Reduces redundancy through normalization - Flexible - relationships can be modified easily using keys - Ensures data integrity through constraints (primary key, foreign key, etc.)

Disadvantages: - Can have slower performance for very large, complex datasets compared to specialized models - Complex relationships (many-to-many) require additional junction tables

Example Systems: MySQL, Oracle, SQL Server, PostgreSQL

Comparison of Data Models

Basis	Hierarchical	Network	Relational
Structure	Tree	Graph	Table
Relationship	One-to-many (parent-child)	Many-to-many (owner-member)	Any (via keys)

Flexibility	Low	Medium	High
Complexity	Simple for hierarchy, complex otherwise	Complex	Simple, user-friendly
Query Language	Navigational (procedural)	Navigational (procedural)	SQL (non-procedural/declarative)
Example	IMS	IDMS	MySQL, Oracle

Quick Revision Points

- **Data** = raw facts; **Information** = processed data
- **DBMS** removes drawbacks of file systems (redundancy, inconsistency, poor security)
- **3-level abstraction**: Physical → Logical → View
- **Data Independence**: Physical (easy) and Logical (harder)
- **Schema** = structure (fixed); **Instance** = actual data (changes)
- **Codd's 12 Rules (0-12)** define a true RDBMS
- **Two-tier** = Client + DB Server; **Three-tier** = Client + App Server + DB Server
- **Hierarchical** = tree, one parent; **Network** = graph, multiple parents; **Relational** = tables, keys